Lab Sheet 3 — Bash Shell Scripting

Complete questions with answers immediately below each question. Commands are written for common Linux distributions; use sudo only when required.

Lab Sheet 3 — Bash Shell Scripting

distributions; use sudo only when required.

Lab Sheet 3 — Bash Shell Scripting

sudo only when required.

Question 1: Hello Script Create hello.sh using a shebang and a variable that prints a personalized greeting. Make it executable and run it using ./hello.sh and bash hello.sh.

Answer / Solution:

```
#!/bin/bash
name="Student"
echo "Hello, $name!"
# Run: chmod +x hello.sh; ./hello.sh; bash hello.sh
```

Question 2: User Input Script Create user_input.sh that asks for a student's name, branch and semester using read, then prints a formatted welcome message.

Answer / Solution:

```
#!/bin/bash
read -r -p "Name: " name
read -r -p "Branch: " branch
read -r -p "Semester: " semester
printf "Welcome %s | Branch: %s | Semester: %s\n" "$name" "$branch" "$semester"
```

Question 3: Arithmetic Calculator Create calculator.sh that reads two integers and displays their sum, difference, product and quotient using Bash arithmetic.

Answer / Solution:

```
#!/bin/bash
read -r -p "First integer: " a
read -r -p "Second integer: " b
echo "Sum: $((a+b))"; echo "Difference: $((a-b))"; echo "Product: $((a*b))"
if (( b != 0 )); then echo "Quotient: $((a/b))"; else echo "Quotient: undefined (division by zero)"; fi
```

Question 4: Command Substitution Create system_info.sh that stores the current date, current working directory and logged-in username in variables using command substitution, then displays them.

Answer / Solution:

```
#!/bin/bash
date_now=$(date)
working_dir=$(pwd)
username=$(whoami)
printf "Date: %s\nDirectory: %s\nUser: %s\n" "$date_now" "$working_dir" "$username"
```

Question 5: Number Classification Create number_type.sh that reads an integer and reports whether it is positive, negative or zero using if/elif/else.

Answer / Solution:

```
#!/bin/bash
read -r -p "Enter an integer: " n
if (( n > 0 )); then echo Positive; elif (( n < 0 )); then echo Negative; else echo Zero; fi
```

Question 6: Even or Odd Create even_odd.sh that accepts one integer as $1 and reports whether it is even or odd. If the argument is missing, display a usage message and exit with a non-zero status.

Answer / Solution:

```
#!/bin/bash
if [[ $# -lt 1 ]]; then echo "Usage: $0 INTEGER" >&2; exit 1; fi
if (( $1 % 2 == 0 )); then echo Even; else echo Odd; fi
```

Question 7: File and Directory Tests Create file_check.sh that accepts a path as $1 and reports whether the path is a regular file, a directory, or neither. Also report whether a regular file is executable when applicable.

Answer / Solution:

```
#!/bin/bash
p=${1:-}
if [[ -f "$p" ]]; then echo "Regular file"; [[ -x "$p" ]] && echo "Executable" || echo "Not executable"
```

elif [[-d "$p"]]; then echo "Directory"

else echo "Neither a regular file nor a directory"; exit 1; fi

Question 8: For Loop Practice Create multiplication.sh that accepts a number as $1 and prints its multiplication table from 1 to 10 using a for loop.

Answer / Solution:

```
#!/bin/bash
if [[ ! $1 =~ ^-?[0-9]+$ ]]; then echo "Usage: $0 INTEGER" >&2; exit 1; fi
for ((i=1;i<=10;i++)); do printf "%d x %d = %d\n" "$1" "$i" "$(( $1*i ))"; done
```

Question 9: Loop Through Files Create list_txt.sh that loops through all .txt files inside the data directory and prints each filename with a short message.

Answer / Solution:

```
#!/bin/bash
shopt -s nullglob
for f in data/*.txt; do echo "Text file: $f"; done
```

Question 10: While Loop Create countdown.sh that uses a while loop to print a countdown from 10 to 1 and then prints 'Done!'.

Answer / Solution:

```
#!/bin/bash
n=10
while (( n >= 1 )); do echo "$n"; ((n--)); done
echo "Done!"
```

Question 11: Script Arguments Create arguments.sh that displays $0, $1, $2, $#, and $@. Test it with at least two different command-line argument sets.

Answer / Solution:

```
#!/bin/bash
printf 'Script: %s\nFirst: %s\nSecond: %s\nCount: %s\n' "$0" "${1-}" "${2-}" "$#"
printf 'All arguments:'; printf ' %q' "$@"; printf '\n'
# Test: ./arguments.sh Amit MCA; ./arguments.sh Neha 2nd Sem
```

Question 12: Exit Codes Create status.sh that accepts a directory name. Exit with 0 when the directory exists and exit with 1 when it does not. After running it, inspect the result using echo $?.

Answer / Solution:

```
#!/bin/bash
if [[ -d ${1:-} ]]; then echo "Directory exists"; exit 0; else echo "Directory does not exist"; exit 1;
fi
# Run: ./status.sh data; echo $?
```

Question 13: Output Redirection Write commands that redirect the listing of the data directory to output/files.txt, first overwriting the file and then demonstrating the difference between > and >>.

Answer / Solution:

```
mkdir -p output && ls data > output/files.txt # overwrite/create
ls data >> output/files.txt # append another listing
# > truncates the file; >> adds to its end.
```

Question 14: Append a Log Create log_entry.sh that appends the current date/time and a user-supplied message to logs/activity.log without overwriting previous entries.

Answer / Solution:

```
#!/bin/bash
mkdir -p logs
read -r -p "Message: " msg
printf '[%s] %s\n' "$(date '+%F %T')" "$msg" >> logs/activity.log
```

Question 15: Input Redirection and Sorting Use data/names.txt and input redirection to sort the names. Extend the command so duplicate names are removed.

Answer / Solution:

```
sort < data/names.txt
```

```
sort -u < data/names.txt # sort and remove duplicate lines
```

Question 16: Error Logging Run a command that deliberately produces an error, redirect only standard error to logs/errors.log, and then display the log.

Answer / Solution:

```
mkdir -p logs && ls /path/that/does/not/exist 2> logs/errors.log
cat logs/errors.log
```

Question 17: Combined Output and Error Logging Run one command containing both a valid and invalid path and redirect standard output and standard error into logs/combined.log. Verify the file.

Answer / Solution:

```
mkdir -p logs && ls data /path/that/does/not/exist > logs/combined.log 2>&1
cat logs/combined.log
```

Question 18: Pipeline Practice Use a pipeline with ps aux and grep to find processes matching a word of your choice. Then create another pipeline that counts the matching lines.

Answer / Solution:

```
ps aux | grep -i '[s]sh'
ps aux | grep -i '[s]sh' | wc -l # replace ssh with the word to search
```

Question 19: Backup Checker Script Create backup_check.sh that accepts a directory as $1, verifies that it exists, scans for .bak files, counts them, reports each backup found, and appends a 'No backup files found' message to logs/backup_log.txt when the count is zero.

Answer / Solution:

```
#!/bin/bash
if [[ ! -d ${1:-} ]]; then echo "Usage: $0 DIRECTORY" >&2; exit 1; fi
mkdir -p logs
shopt -s nullglob
files=("$1"/*.bak)
if (( ${#files[@]} == 0 )); then echo "No backup files found in $1" | tee -a logs/backup_log.txt
else printf 'Backup found: %s\n' "${files[@]}"; echo "Count: ${#files[@]}"; fi
```

Question 20: Integrated File Report Create file_report.sh that accepts exactly one filename, verifies that it exists and is readable, displays its line/word/character counts using wc, and appends a timestamped processing record to logs/execution_history.log. Test both successful and failure cases.

Answer / Solution:

```
#!/bin/bash
if [[ $# -ne 1 ]]; then echo "Usage: $0 FILENAME" >&2; exit 1; fi
mkdir -p logs
f=$1
if [[ ! -f "$f" || ! -r "$f" ]]; then echo "File does not exist or is not readable: $f" >&2; exit 1; fi
wc -l -w -m "$f"
printf '[%s] processed %s\n' "$(date '+%F %T')" "$f" >> logs/execution_history.log
# Test success: ./file_report.sh data/notes.txt; failure: ./file_report.sh missing.txt
```